Estructura de Dades: Pràctica 3

Pràctica 3. Estructures de dades no lineals: Arbres binaris

Introducció

Objectius: Familiaritzar-se amb les estructures no lineals, concretament, amb els arbres binaris de cerca i els arbres binaris balancejats, analitzant les seves característiques i diferències.
Tema de teoria relacionat amb la pràctica: Tema 4 Estructures no lineals: Arbres

Enunciat

Es demana implementar una aplicació per la indexació de paraules en un text. Donat un text qualsevol es pretén construir una estructura de dades que permeti cercar paraules en el text de forma eficient. Aquesta estructura haurà de mantenir informació sobre les posicions en les que surt cada paraula, la paraula en el text i permetre accedir a aquesta informació de manera ràpida. Per exemple:

```
1  Binary search tree
2
3  From Wikipedia, the free encyclopedia
4
5  In computer science, binary search trees (BST), sometimes called ordered
   or sorted binary trees, are a class of data structures used to implement
   lookup tables and dynamic sets. They store data items, known as keys,
   and allow fast insertion and deletion of such keys, as well as checking
   whether a key is present in a tree.
6
7  Binary search trees keep their keys in sorted order, so that lookup and
   other operations can use the principle of binary search: when looking for
   a key in a tree (or a place to insert a new key), they traverse the tree
   from root to leaf, making comparisons to keys stored in the nodes of
   the tree and deciding, based on the comparison, to continue searching
   in the left or right subtrees. On average, this means that each compa-
   rison allows the operations to skip over half of the tree, so that each
   lookup/insertion/deletion takes time proportional to the logarithm of the
   number of items stored in the tree. This is much better than the linear
   time required to find items by key in an unsorted array, but slower than
   the corresponding operations on hash tables.
```

En el text anterior, si es cerqués la paraula *binary*, s'haurien d'obtenir els següents resultats, tenint en compte que entre la primera i segona línia de text hi ha una línia en blanc:

- Línia 1, paraula 1
- Línia 5, paraula 4
- Línia 5, paraula 13
- Línia 7, paraula 1
- Línia 7, paraula 21

Estructura de Dades: Pràctica 3

En aquesta pràctica es demana implementar aquesta funcionalitat utilitzant arbres de cerca binària.

Exercicis

Consideracions prèvies importants per a la realització dels exercicis:

- El **codi s'ha de comentar obligatòriament**:
  a) Als fitxers de declaració (.h), hi haureu d'especificar per cada operació del TAD que implementareu el cost computacional teòric en el pitjor dels casos. També heu d'incloure comentaris a les parts més importants de les funcions que implementeu per facilitar la lectura del codi.
  b) Als fitxers d'implementació (.cpp), hi haureu d'incloure els comentaris necessaris per facilitar la lectura del codi.
- **Controleu errors** quan ho considereu oportú, llençant les degudes excepcions i caçant-les a les seves corresponents clàusules try-catch.
- La **implementació amb templates NO és opcional, és obligatori en aquesta pràctica**.
- Podeu implementar mètodes de suport addicionals, sempre que siguin necessaris pel desenvolupament de la pràctica, tot i justificant al codi el seu ús i el seu cost computacional teòric.

Exercici 1. Arbre binari de cerca

Implementareu, el TAD **`BinaryTree`** que representi l'arbre binari de cerca amb nodes encadenats. Els nodes de l'arbre seran cadascun un TAD **`Position`**, que també haureu d'implementar i que representarà una clau i un vector d'elements. A continuació es presenten les definicions de les especificacions de cada classe. Més endavant teniu la descripció dels passos per implementar els TADs i les explicacions del que ha de fer cadascuna de les operacions de cada TAD:

BinaryTree.h

```
template <class Key, class Value>
class BinaryTree {
public:
    BinaryTree();
    BinaryTree(const BinaryTree<Key, Value>& orig);
    virtual ~BinaryTree();
    bool isEmpty() const;
    Position<Key, Value>* getRoot() const;
    int size() const;
    int height() const;
    virtual Position<Key, Value>* insert(const Key& key, const Value& value);
    bool contains(const Key& key) const;
    const vector<Value>& getValues(const Key& key) const;
    void printPreOrder(const Position<Key, Value> *node = nullptr) const;
    void printPostOrder(const Position<Key, Value> *node = nullptr) const;
    bool identicalTree(const BinaryTree<Key, Value>& other) const;

protected:
    Position<Key, Value>* root;
private:
    int _size;
    /* Mètodes auxiliars definiu aquí els que necessiteu */
};
```

Position.h

```
template <class Key, class Value>
class Position {
public:
    Position(const Key key);
    Position(const Position<Key, Value>& orig);
    virtual ~Position();
    /* Modificadors */
    // Declareu-hi aquí els modificadors (setters) dels atributs que manquen

    /* Consultors */
    const Key& getKey() const;
    const vector<Value>& getValues() const;
    // Declareu-hi aquí els consultors (getters) dels atributs que manquen

    /* Operacions */
    bool isRoot() const;
    bool isLeaf() const;
    int depth() const;
    int height() const;
    void addValue(const Value& value);
    bool operator==(const Position<Key, Value>& other) const;

private:
    Key key;
    vector<Value> values;
    // Afegiu-hi aquí els atributs que manquen
};
```

Pas 1 Implementar el position de l'arbre binari de cerca

Noteu que el **TAD Position** per al problema que tractarem més endavant guardarà una clau que anomenarem *key* i un vector de *values* que permetran guardar un conjunt de valors associats a la clau. A continuació teniu la descripció de les operacions del TAD **`Position`** (sense tenir en compte els consultors i modificadors dels atributs que falten per especificar):

Operació	Definició
`constructor`	construeix una position (un nou node) de l'arbre binari, passant com a paràmetre una clau
`Constructor còpia`	construeix una còpia de la position a partir de la position rebuda per paràmetre
`destructor`	destrueix els nodes fills. *Atenció a la gestió de memòria dinàmica*
`depth`	calcula recursivament la profunditat de la position. Retorna 0 si es el root.
`height`	retorna l'alçada de la position
`getKey`	retorna la clau de la position (atribut `Key`)
`getValues`	retorna un vector amb tots els valors de la position (atribut `Values`)
`parent`	retorna l'adreça del pare d'una position

`left`	retorna l'adreça del fill esquerra de la position
`right`	retorna l'adreça del fill dret de la position
`setParent`	modifica l'adreça del pare de la position
`setLeft`	modifica l'adreça del fill esquerra de la position
`setRight`	modifica l'adreça del fill dret de la position
`isRoot`	retorna cert si la position és el root de l'arbre, fals en cas contrari
`isLeaf`	retorna cert si la position és una fulla de l'arbre, fals en cas contrari
`addValue`	Afegeix al vector de values un nou valor passat com a paràmetre
`operator==`	(Sobrecarrega de l'operador d'igualtat) Retorna cert si dos nodes són iguals: tenen la mateixa clau i els mateixos valors

<u>Pas 2</u> Implementar l'arbre binari de cerca

Implementeu el TAD **BinaryTree** (obligatori fer-ho amb templates), que representi l'arbre binari amb una representació encadenada. Teniu en compte que cada node de l'arbre estarà representat per una paraula on hi guardarem un vector de tuples que contindrà la línia i posició de la paraula en el text. Únicament s'han d'implementar aquells mètodes que siguin necessaris pel desenvolupament de la pràctica. La descripcions de les operacions del TAD **`BinaryTree`** són les següents:

Operació	Definició
`constructor`	Construeix l'arbre buit.
`destructor`	Destrueix tots els elements de l'arbre binari. *Atenció a la gestió de memòria dinàmica*
`isEmpty`	Retorna cert si l'arbre binari està buit, fals en cas contrari
`getRoot`	Retorna l'adreça del root
`size`	Retorna el nombre de nodes que hi ha l'arbre binari. Aquesta funció recorre els nodes per calcular quants nodes té l'arbre binari.
`height`	Retorna un enter amb l'alçada de l'arbre binari de cerca. **Aquesta funció s'ha d'implementar de forma RECURSIVA**
`insert`	Afegeix un nou node a l'arbre binari de cerca
`contains`	Retorna cert si l'arbre binari conté la Key, fals en cas contrari. **Aquesta funció s'ha d'implementar de forma ITERATIVA**
`getValues`	Retorna un vector amb tots els valors associats a una clau. La funció llença una excepció si no es troba la clau a l'arbre
`printPreorder`	Mostra per consola el contingut l'arbre seguint un recorregut en preordre
`printPostorder`	Mostra per consola el contingut l'arbre seguint un recorregut en postordre
`identicalTree`	Retorna cert si l'arbre binari intern és igual a l'arbre binari que es passa per paràmetre, fals en cas contrari
`search`	Cerca un element de l'arbre binari de cerca, retorna la position si la troba, nullptr en cas contrari. **Aquesta funció s'ha d'implementar de forma RECURSIVA.**

Al codi (en el fitxer BinaryTree.h i Position.h) justifiqueu el cost computacional teòric de les funcions del TAD **BinaryTree** i del TAD **Position**. Es poden implementar altres mètodes que siguin necessaris pel desenvolupament de la pràctica, tot i justificant al codi el seu ús i el seu cost computacional teòric.
Llenceu les excepcions oportunes a cada TAD. El codi s'ha de comentar obligatòriament.

Pas 3 Programació d'un main amb un cas de prova

Per a comprovar la correcta implementació dels TADs, creeu finalment un `main.cpp` que realitzi les operacions del cas de prova que us deixem a continuació i compareu els resultats obtinguts pel vostre codi amb els resultats que us proporcionem en el cas. En aquesta prova, les claus i els valors seran de tipus enter (`int`).

Cas de Prova

```
void mainExercici1(){
    BinaryTree<int, int> tree1;
    int testKeys[] = {2, 0, 8, 45, 76, 5, 3, 40};
    int testValues[] = {5, 5, 1, 88, 99, 12, 9, 11};

    for (int i = 0; i < 8 ; i++) {
        cout << "Inserta a l'arbre la key " << testKeys[i] << " amb valor " <<
testValues[i] << endl;
        tree1.insert(testKeys[i], testValues[i]);
    }

    cout << "Preorder = [";
    tree1.printPreOrder();
    cout << "]" << endl;

    cout << "Postorder = [";
    tree1.printPostOrder();
    cout << "]" << endl;

    BinaryTree<int, int> tree2(tree1);
    cout << " identics " << tree1.identicalTree(tree2) << endl;

    cout << "Preorder = [";
    tree1.printPreOrder();
    cout << "]" << endl;


    tree2.insert(4, 4);
    cout << " identics " << tree2.identicalTree(tree1) << endl;

    cout << "Preorder = [";
    tree2.printPreOrder();
    cout << "]" << endl;

}
```

```
OUTPUT:

Inserta a l'arbre la key 2 amb valor 5

Inserta a l'arbre la key 0 amb valor 5

Inserta a l'arbre la key 8 amb valor 1

Inserta a l'arbre la key 45 amb valor 88

Inserta a l'arbre la key 76 amb valor 99

Inserta a l'arbre la key 5 amb valor 12

Inserta a l'arbre la key 3 amb valor 9

Inserta a l'arbre la key 40 amb valor 11

Preorder = [2 0 8 5 3 45 40 76

]

Postorder = [0 3 5 40 76 45 8 2

]

 identics 1

Preorder = [2 0 8 5 3 45 40 76

]

 identics 0

Preorder = [2 0 8 5 3 4 45 40 76

]
```

Exercici 2. Cercador de paraules amb arbres de cerca binària

En aquest exercici cal implementar un cercador de paraules utilitzant un arbre de cerca binària. Per a resoldre el problema seguiu els passos següents:

Pas 1. Especificació de Tuple

Implementeu el TAD **`Tuple`** que contingui totes les dades necessàries per emmagatzemar la informació de la línia i el lloc de la paraula en aquesta línia. El TAD **`Tuple`** serà, en aquest exercici, el contingut dels valors que hi ha a les positions de l'arbre de cerca binària. L'especificació mínima del TAD és la següent:

Operació	Definició
`constructor`	construeix una Tuple amb totes les dades
`funcions consultores`	una o més funcions per consultar cada una de les dades associades a la Tuple
`funcions modificadores`	una o més funcions per modificar cada una de les dades associades a la Tuple

Sobrecarrega operadors	fareu la sobrecarrega de l'operator< i operator==

Pas 2. Implementació del Cercador de paraules amb un arbre

Escriviu l'especificació d'una classe anomenada **`WordIndexer`** que conté un objecte de tipus **BSTree**. La classe WordIndexer es cridarà des del main i ha de realitzar les següents operacions:

Operació	Definició
`Constructors`	Constructor per defecte, constructor amb paràmetres que rebi el nom del fitxer que ha d'obrir, constructor còpia
`Destructor`	Destructor de la classe
`Contains(word)`	Rep una paraula i retorna cert en cas que aquesta paraula estigui a l'arbre binari i fals en cas contrari
`printOccurrences(word)`	Aquest mètode rep una paraula i retorna un vector de tuples que conté les línies i posicions de la paraula en el text
`printDictionary`	Aquest mètode mostra per pantalla l'índex de paraules que surten a l'arbre en ordre alfabètic, junt amb les posicions on apareixen.
`addText (filename)`	Aquest mètode rep el nom d'un fitxer i emmagatzema el seu contingut a dins de l'arbre. El contingut es guarda en minúscules. Els signes de puntuació s'han d'eliminar de les paraules.
`insertWord(word, line, positionWord)`	Aquest mètode rep el nom d'una paraula, la línia i la posició de la paraula on apareix en el text i fa la inserció a l'arbre. Teniu en compte que l'arbre no té elements repetits, en cas que ja existeixi haureu d'actualitzar el contingut de la Position (node de l'arbre).

La definició de l'especificació de la classe és doncs:

```
class WordIndexer {
public:
    WordIndexer();
    WordIndexer(std::string path);
    WordIndexer(const WordIndexer& orig);
    virtual ~WordIndexer();
    int size() const;
    int height() const;
    bool contains(const std::string &word) const;
    void printOccurrences(const std::string &word) const;
    void printDictionary(Position<string, Tuple<int> > *node = nullptr) const;
protected:
    BinaryTree<string, Tuple<int> > *tree;
    void addText(std::string path);
private:
    void insertWord(const std::string &word, const int &line, const int &position);
     /* Metodes auxiliars, definiu-los aquí sota */
};
```

Pas 3. Programació del main

A l'algorisme principal s'han de realitzar les següents accions:

1. Demanar a l'usuari el nom del fitxer que volem utilitzar amb la pregunta: "Quin fitxer vols (P/G)?" (per petit o gran). Al campus virtual trobareu dos fitxers de text que podeu utilitzar per fer les proves: *`shortText.txt`* i *`longText.txt`*. Aquests fitxers s'hauran d'incorporar a la carpeta del projecte i incloure'ls com a recursos en el projecte. Avaluar el temps d'inserció i mostrar-lo al final de tot. En aquest apartat, a més a més de demanar el nom del fitxer i llegir-lo, cal:
   - Crear un objecte `WordIndexer`.
   - Inicialitzar el `WordIndexer` a partir del contingut del fitxer.
   - Mostrar el temps de creació del `WordIndexer`.
2. Mostrar l'arbre segons l'ID en ordre creixent. Fer un comptador que demani confirmació cada 40 paraules per seguir mostrant l'arbre.
3. Llegir el fitxer *`dictionary.txt`* que us proporcionem per testejar l'aplicació. Per a cada paraula continguda al fitxer, fer una cerca en l'arbre emmagatzemat a `WordIndexer`. Avaluar el temps de cerca de totes les paraules del fitxer.
4. Generar i visualitzar l'índex de les paraules.
5. Visualitzar per pantalla la profunditat de l'arbre guardat en WordIndexer.

Implementeu aquestes funcionalitats en forma d'un menú al main que permeti realitzar les 5 accions descrites anteriorment i l'opció 6 serà sortir del menú.

Atenció: En aquest exercici és opcional fer la implementació amb templates.

Generació d'un índex de paraules. Un índex de paraules és un llistat complet de les paraules que surten en un text en ordre alfabètic, junt amb les posicions a on hi surten. Per exemple, pel text mostrat a la pàgina 1, l´índex començaria així:

- bst [(5,7)]
- or [(5, 11), (7, 31)]
- a [(5, 16), (5, 50), (5, 55), (7, 26), (7, 29), (7, 32), (7, 36)]
- allow [(5, 37)]
- allows [(7, 80)]
- an [(7, 124)]
- and [(5, 26), (5, 36), (5, 40), (7, 13), (7, 58)]
- ...
- binary [(1, 1), (5, 4), (5, 13), (7, 1), (7, 21)]
- ...

Mesureu el temps total que es triga en generar el llistat de paraules. Per mesurar els temps, podeu fer servir l'alternativa que vulgueu. Per tenir precisió inferior a segons, per exemple, mil·lèsimes de segon (ms), una bona opció pot ser la llibreria **std::chrono**:

```
#include <chrono>
#include <iostream>
using namespace std;
```

```
chrono::steady_clock::time_point begin = chrono::steady_clock::now();
// Aquí el vostre codi del que en voleu mesurar el temps d'execució
chrono::steady_clock::time_point end = chrono::steady_clock::now();

cout << "Temps transcorregut: " << chrono::duration_cast<chrono::seconds>(end -
begin).count() << " s." << endl;
```

Lliurament

A partir de la descripció del problema, es demana:

- Implementar els exercicis en **Visual Studio Code i C++ versió 11**. Lliurar el codi C++ corresponent als vostres exercicis en una carpeta anomenada codi, amb tot el codi dels TADs i el main.cpp. Noteu que hi ha només 1 main, que conté el codi per provar cadascun dels exercicis.
- No us oblideu d'afegir a la carpeta principal els fitxers de les proves: shortText.txt, longText.txt, unbalancedText.txt i el dictionary.txt.

Com a màxim el dia del lliurament es penjarà en el campus virtual un fitxer comprimit **en format ZIP** amb el nom i cognoms de l'alumne, el nom del grup (A, B, C, D, E o F) i el numero de la pràctica com a nom de fitxer, **GrupA_BartSimpson_P3.zip**, on A indica grup de pràctiques A i P3 indica que és la "pràctica 3". El fitxer ZIP inclourà la carpeta codi.

Els criteris per acceptar la pràctica són:

- La pràctica ha de funcionar en la seva totalitat.
- La pràctica ha de ser orientada a objectes.
- El codi ha d'estar comentat.
- La implementació ha de garantir una estructura clara i un codi optimitzat, seguint les bones pràctiques de programació.

IMPORTANT: La còpia de la implementació d'una pràctica implica **un zero a la nota global de pràctiques** de l'assignatura per les persones implicades (tant la que ha copiat com la que ha deixat copiar).

Planificació

Del 14 d'abril al 17 de maig 2026. Per aquesta pràctica els professors proposen la següent planificació:

- **Setmana 1** *(Classe de Laboratori 8)*
  - Exercici 1. Implementació del **BinaryTree**, el main amb el cas de prova inclòs, i comentar el codi
- **Setmana 2** *(Classe de Laboratori 9)*
  - Exercici 2. Implementació de la classe **WordIndexer** i del main.cpp amb el menú, i comentar el codi
- **Setmana 3** *(Classe de Laboratori 10)*
  - Exercici 3. Implementació de la classe **BalancedTree**, el main amb el cas de prova inclòs, i comentar el codi
- **Setmana 4** *(Classe de Laboratori 11)*
  - Exercici 4. Implementació de la classe **WordIndexerBalanced** i del main.cpp amb el menú, i comentar el codi

- Exercici 5. Avaluació de les estructures

- **Setmana 5** *(Classe de Laboratori 12)*
  - **PROVA DE LA PRÀCTICA 3 al laboratori.**

Recordeu que la setmana 4 s'ha de lliurar la totalitat de la pràctica (tots els exercicis al ZIP).

Lliurament: dia **17 de maig de 2026**